Student Course Management System

COS 201 - Computer Programming I | Lab Assessment

Name: Muhammad SaniAbdulkadir

Matriculation number: 2025/B/CYB/0948

Student ID: 301826989

Email: m.abdulkadir6989@miva.edu.ng

Department: Cyber Security

What the system does

This Java console program keeps a list of courses for a semester. Each course has a code, a title and credit units. The menu lets the user add a course, view the list, search by code, calculate total units, save the list, load a saved list and exit.

How the main features work

The Course class stores and validates one course. CourseManager keeps the courses in an ArrayList and provides the search, total, save and load methods. StudentCourseApp reads input with Scanner and uses a loop and switch statement to handle the seven menu choices.

Course codes are trimmed, converted to uppercase and checked for valid characters. Titles are trimmed and cannot be blank. Units must be whole numbers from 1 to 10; this is a program validation rule. Duplicate course codes are rejected. Searching ignores the letter case used by the user.

Saving writes a UTF-8 text file with the header COURSES_V1 and one tab-separated record per line. The default filename is courses.tsv. Loading replaces the current list only after every record is valid. The menu asks before overwriting an existing file or replacing unsaved changes.

Recursion

The totalUnits method calls sumUnits(start, end). An empty range returns zero, and a range with one course returns that course's units. Larger ranges are split in half, and the two recursive results are added. This reduces the depth of the call stack. Loops are used for the menu, displaying courses, searching and processing file records.

Error handling and validation

The main error cases are non-numeric units, blank fields, duplicate codes, missing files and damaged file records. NumberFormatException handles invalid numeric input. IllegalArgumentException reports invalid course details. IOException reports file failures. A failed load leaves the current courses intact. If input closes, the program exits and warns when changes have not been saved.

Testing results

The automated checks passed: 33 core checks and 30 console checks. They cover course validation, normalized search, duplicate detection, recursive totals, saving and loading, empty lists, UTF-8 text, malformed files, missing paths, menu errors, cancellation and closed input. A 10,000-course fixture also produced the expected total without a stack overflow.

A separate run used the eight courses shown in the supplied course list. It displayed all eight, found COS 201 using lowercase input, calculated 18 units, saved the file and loaded eight courses back. The image below is a browser screenshot of the actual captured console output; the full output is included in program-transcript.txt. These checks demonstrate the tested cases, rather than guaranteeing every possible input.



Run: bash scripts/build.sh, then java -cp build StudentCourseApp. Tests: bash scripts/test.sh. Requires Java 21; the cloud environment also has the Eclipse compiler available.

Source Code Appendix

Student Course Management System - application source files

Course.java

```
import java.util.Locale;

/** One validated course. */
public final class Course {
    private final String code;
    private final String title;
    private final int units;

    public Course(String code, String title, int units) {
        this.code = normalizeCode(code);
        this.title = title == null ? "" : title.trim();
        if (!this.code.matches("[A-Z][A-Z0-9 -]{1,29}")) {
            throw new IllegalArgumentException("Code must be 2–30 letters, digits, spaces or hyphens and
start with a letter.");
        }
        if (this.title.isEmpty() || this.title.length() > 120
                || this.title.chars().anyMatch(Character::isISOControl)) {
            throw new IllegalArgumentException("Title must contain 1–120 characters without tabs or line
breaks.");
        }
        if (units < 1 || units > 10) {
            throw new IllegalArgumentException("Units must be between 1 and 10.");
        }
        this.units = units;
    }

    public static String normalizeCode(String code) {
        return code == null ? "" : code.trim().toUpperCase(Locale.ROOT).replaceAll(" +", " ");
    }

    public String getCode() { return code; }
    public String getTitle() { return title; }
    public int getUnits() { return units; }

    @Override
    public String toString() {
        return String.format("%-30s | %-50s | %d", code, title, units);
    }
}
```

CourseManager.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.List;

/** Stores courses and handles UTF-8 tab-separated persistence. */
public final class CourseManager {
    private static final String HEADER = "COURSES_V1";
    private List<Course> courses = new ArrayList<>();

    public void add(Course course) {
        if (find(course.getCode()) != null) {
            throw new IllegalArgumentException("A course with this code already exists.");
        }
        courses.add(course);
    }

    public List<Course> getCourses() { return List.copyOf(courses); }

    public Course find(String code) {
        String normalized = Course.normalizeCode(code);
        for (Course course : courses) {
            if (course.getCode().equals(normalized)) { return course; }
        }
        return null;
    }

    public int totalUnits() { return sumUnits(0, courses.size()); }

    // Divide the range recursively; logarithmic depth avoids a long call stack.
    private int sumUnits(int start, int end) {
        if (start == end) { return 0; }
        if (end - start == 1) { return courses.get(start).getUnits(); }
        int middle = start + (end - start) / 2;
        return sumUnits(start, middle) + sumUnits(middle, end);
    }

    public void save(Path path) throws IOException {
        List<String> lines = new ArrayList<>();
        lines.add(HEADER);
        for (Course course : courses) {
            lines.add(course.getCode() + "\t" + course.getTitle() + "\t" + course.getUnits());
        }
        Path target = path.toAbsolutePath();
        Path temporary = Files.createTempFile(target.getParent(), "courses-", ".tmp");
        try {
            Files.write(temporary, lines, StandardCharsets.UTF_8);
            try {
                Files.move(temporary, target, StandardCopyOption.REPLACE_EXISTING,
                        StandardCopyOption.ATOMIC_MOVE);
            } catch (AtomicMoveNotSupportedException exception) {
                Files.move(temporary, target, StandardCopyOption.REPLACE_EXISTING);
            }
        } finally {
            Files.deleteIfExists(temporary);
        }
    }

    public void load(Path path) throws IOException {
        List<String> lines = Files.readAllLines(path, StandardCharsets.UTF_8);
        if (lines.isEmpty() || !HEADER.equals(lines.get(0))) {
            throw new IOException("Invalid course file header.");
        }
        CourseManager loaded = new CourseManager();
        for (int index = 1; index < lines.size(); index++) {
            String[] fields = lines.get(index).split("\t", -1);
            try {
                if (fields.length != 3) {
                    throw new IllegalArgumentException("Expected code, title and units.");
```

```
                }
                loaded.add(new Course(fields[0], fields[1], Integer.parseInt(fields[2])));
            } catch (IllegalArgumentException exception) {
                throw new IOException("Invalid data on line " + (index + 1) + ": "
                        + exception.getMessage(), exception);
            }
        }
        courses = loaded.courses; // Replace only after the entire file passes validation.
    }
}
```

StudentCourseApp.java

```
import java.io.IOException;
import java.nio.file.InvalidPathException;
import java.nio.file.Path;
import java.util.Scanner;

/** Organized console menu for the Student Course Management mini-system. */
public final class StudentCourseApp {
    private final Scanner input;
    private final CourseManager manager = new CourseManager();
    private boolean changed;

    public StudentCourseApp(Scanner input) { this.input = input; }

    public static void main(String[] args) {
        new StudentCourseApp(new Scanner(System.in)).run();
    }

    private String read(String prompt) {
        System.out.print(prompt);
        return input.hasNextLine() ? input.nextLine().trim() : null;
    }

    public void run() {
        System.out.println("STUDENT COURSE MANAGEMENT SYSTEM");
        while (true) {
            System.out.println("\n1. Add Course\n2. View All Courses\n3. Search Course by Code"
                    + "\n4. Compute Total Units\n5. Save to File\n6. Load from File\n7. Exit Program");
            String choice = read("Select an option: ");
            if (choice == null) {
                System.out.println("\nInput closed." + (changed ? " Unsaved changes were not saved." :
"));
                return;
            }
            try {
                switch (choice) {
                    case "1" -> addCourse();
                    case "2" -> viewCourses();
                    case "3" -> searchCourse();
                    case "4" -> System.out.println("Total units: " + manager.totalUnits());
                    case "5" -> saveCourses();
                    case "6" -> loadCourses();
                    case "7" -> {
                        if (!changed || "y".equalsIgnoreCase(read("Exit without saving changes? (y/N):
"))) {
                            System.out.println("Goodbye.");
                            return;
                        }
                    }
                    default -> System.out.println("Invalid selection. Enter a number from 1 to 7.");
                }
            } catch (NumberFormatException exception) {
                System.out.println("Invalid units. Enter a whole number from 1 to 10.");
            } catch (IllegalArgumentException exception) {
                System.out.println("Invalid input: " + exception.getMessage());
            } catch (IOException exception) {
                System.out.println("File operation failed: " + exception.getMessage());
            }
        }
    }

    private void addCourse() {
        String code = read("Course code: ");
        if (code == null) { return; }
        String title = read("Course title: ");
        if (title == null) { return; }
        String units = read("Units (1–10): ");
        if (units == null) { return; }
        manager.add(new Course(code, title, Integer.parseInt(units)));
        changed = true;
        System.out.println("Course added.");
    }

    private void viewCourses() {
```

```
        if (manager.getCourses().isEmpty()) {
            System.out.println("No courses recorded.");
            return;
        }
        System.out.printf("%-30s | %-50s | Units%n", "CODE", "TITLE");
        for (Course course : manager.getCourses()) { System.out.println(course); }
    }

    private void searchCourse() {
        String code = read("Course code to search: ");
        if (code == null) { return; }
        Course course = manager.find(code);
        System.out.println(course == null ? "Course not found." : course);
    }

    private Path filePath() throws InvalidPathException {
        String value = read("File path (Enter for courses.tsv): ");
        return value == null ? null : Path.of(value.isEmpty() ? "courses.tsv" : value);
    }

    private void saveCourses() throws IOException {
        Path path = filePath();
        if (path == null) { return; }
        if (java.nio.file.Files.exists(path)
                && !"y".equalsIgnoreCase(read("Overwrite existing file? (y/N): "))) {
            System.out.println("Save cancelled.");
            return;
        }
        manager.save(path);
        changed = false;
        System.out.println("Saved " + manager.getCourses().size() + " course(s) to " + path);
    }

    private void loadCourses() throws IOException {
        if (changed && !"y".equalsIgnoreCase(read("Replace unsaved courses? (y/N): "))) {
            System.out.println("Load cancelled.");
            return;
        }
        Path path = filePath();
        if (path == null) { return; }
        manager.load(path);
        changed = false;
        System.out.println("Loaded " + manager.getCourses().size() + " course(s).");
    }
}
```